

БГУИР

Кафедра ЭВМ

Отчет по лабораторной работе №3

Тема: «Разработка тестовых модулей на языке VHDL в среде Vivado»

Выполнил:

студент группы 250504 Казаченко П.Е.

Проверил:

ассистент каф. ЭВМ \_\_\_\_\_ Шеменков В.В.

Минск

2025

## **1. Цель работы**

Цель работы: приобрести навыки разработки тестовых модулей на языке VHDL.

## **2. Исходные данные**

Для комбинационного устройства, разработанного в лабораторной работе №1 (вариант с логическими операциями и параллельным оператором без условного присваивания), создать два варианта тестовых модулей со встроенной проверкой корректности работы устройства.

В первом варианте исходные воздействия, подаваемые на разработанное устройство, и эталонные результаты работы устройства должны считываться тестовым модулем из текстового файла. Текстовый файл может создаваться любым способом (в текстовом редакторе, программой на языке VHDL, C и т.д.).

Во втором варианте входные тестовые воздействия могут формироваться любым способом (считываться из текстового файла, формироваться непосредственно в тестовом модуле и т.д.), а для формирования эталонных воздействий в качестве эталона необходимо использовать любой другой вариант описания этого же устройства из лабораторной работы №1.

Тестовые воздействия в обоих вариантах должны быть полными. Для комбинационного устройства, разработанного в лабораторной работе №1, это означает полный перебор входных значений. В случае обнаружения ошибки в работе устройства необходимо выдать сообщение на консоль программы моделирования.

Для функционального узла последовательного типа, разработанного в лабораторной работе №2, создать тестовый модуль с автоматической проверкой корректности работы устройства. Тестовое воздействие может формироваться любым способом и должно быть по возможности полным, то есть тестировать работу устройства во всех его режимах. В случае обнаружения ошибки в работе устройства необходимо выдать сообщение на консоль программы моделирования.

## **3. Теоретические сведения**

Согласно варианту №9, в лабораторной работе №1 необходимо было разработать мультиплексор, соответствующий логической схеме, представленной на рисунке 3.1.

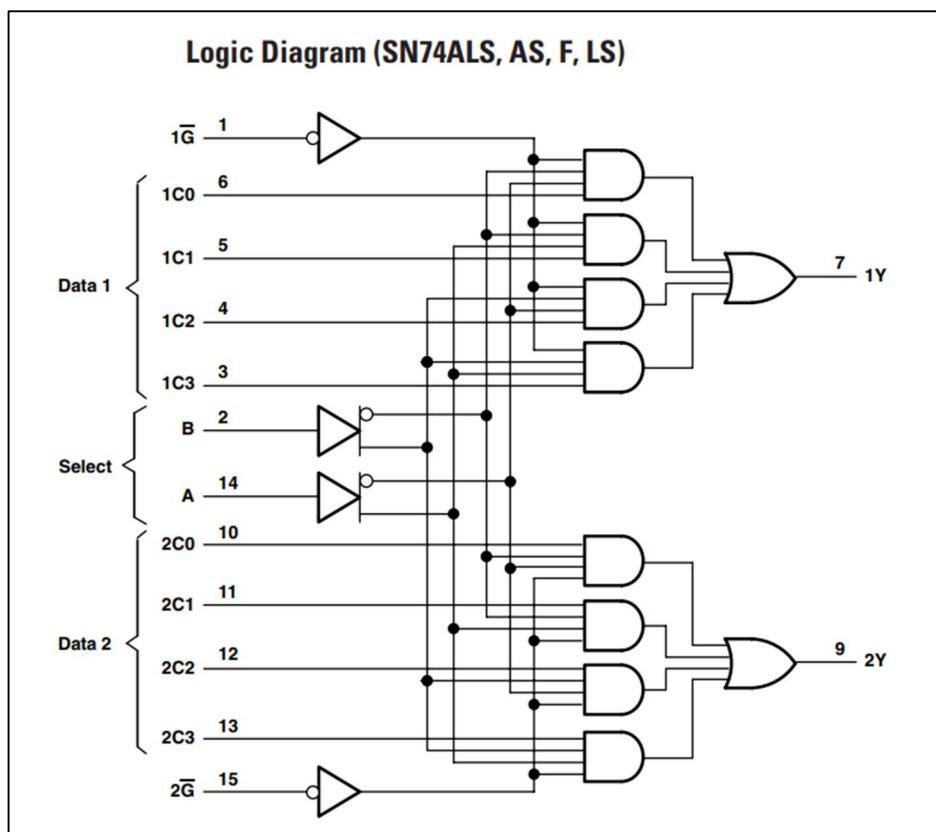


Рис. 3.1 – Логическая схема комбинационного устройства

Функциональная таблица комбинационного устройства представлена на рисунке 3.2.

| FUNCTION TABLE (SN74) |   |             |    |    |    |                |         |
|-----------------------|---|-------------|----|----|----|----------------|---------|
| SELECT INPUTS         |   | DATA INPUTS |    |    |    | STROBE         | OUTPUTS |
| B                     | A | C0          | C1 | C2 | C3 | $\overline{G}$ | Y       |
| X                     | X | X           | X  | X  | X  | H              | L       |
| L                     | L | L           | X  | X  | X  | L              | L       |
| L                     | L | H           | X  | X  | X  | L              | H       |
| L                     | H | X           | L  | X  | X  | L              | L       |
| L                     | H | X           | H  | X  | X  | L              | H       |
| H                     | L | X           | X  | L  | X  | L              | L       |
| H                     | L | X           | X  | H  | X  | L              | H       |
| H                     | H | X           | X  | X  | L  | L              | L       |
| H                     | H | X           | X  | X  | H  | L              | H       |

Рис. 3.2 – Функциональная таблица комбинационного устройства

В лабораторной работе №2 необходимо было разработать регистр, соответствующий логической схеме, представленной на рисунке 3.3.

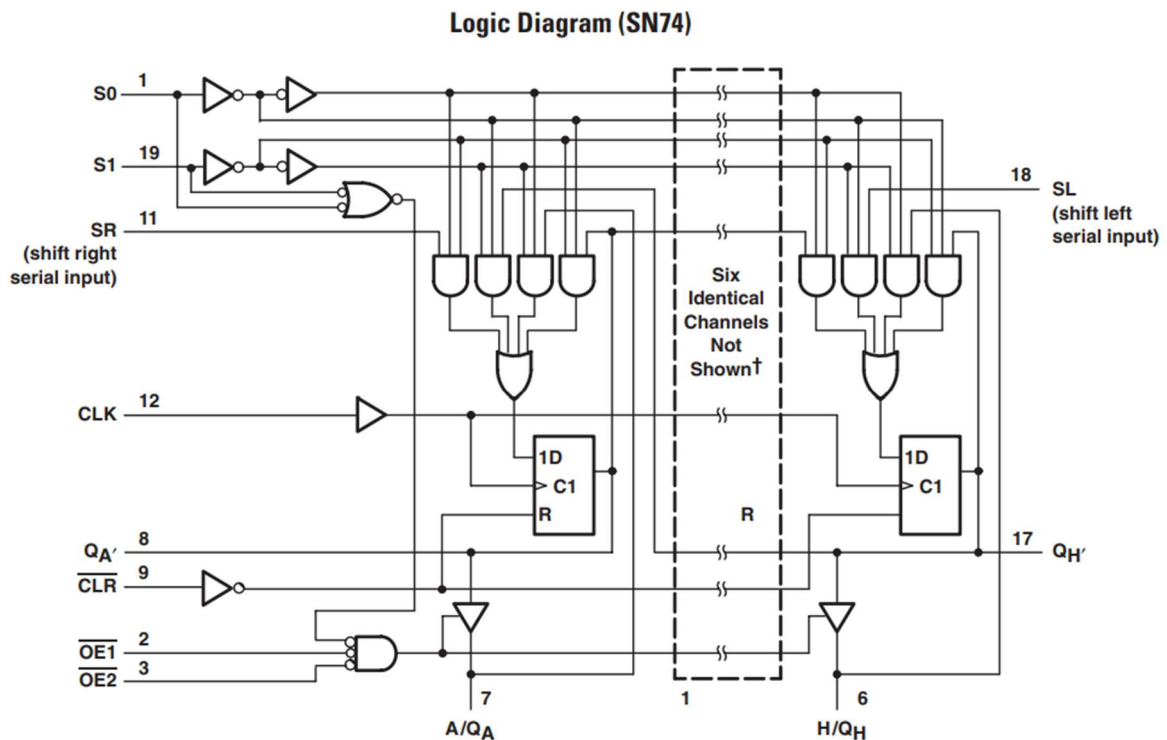


Рис. 3.3 – Логическая схема последовательного устройства  
Функциональная таблица последовательного устройства представлена на рисунке 3.4.

**FUNCTION TABLE (SN74)**

| MODE        | INPUTS |    |    |      |      |     |    |    | I/O PORTS |       |       |       |       |       |       |       | OUTPUTS |     |
|-------------|--------|----|----|------|------|-----|----|----|-----------|-------|-------|-------|-------|-------|-------|-------|---------|-----|
|             | CLR    | S1 | S0 | OE1† | OE2† | CLK | SL | SR | A/Q A     | B/Q B | C/Q C | D/Q D | E/Q E | F/Q F | G/Q G | H/Q H | QA'     | QH' |
| Clear       | L      | X  | L  | L    | L    | X   | X  | X  | L         | L     | L     | L     | L     | L     | L     | L     | L       | L   |
|             | L      | H  | H  | X    | X    | X   | X  | X  | L         | L     | L     | L     | L     | L     | L     | L     | L       | L   |
| Hold        | H      | L  | L  | L    | L    | X   | X  | X  | QA0       | QB0   | QC0   | QD0   | QE0   | QF0   | QG0   | QH0   | QA0     | QH0 |
| Shift Right | H      | L  | H  | L    | L    | ↑   | X  | H  | L         | QA0   | QB0   | QC0   | QD0   | QE0   | QF0   | QH0   | QA0     | QH0 |
| Shift Left  | H      | H  | L  | L    | L    | ↑   | H  | X  | L         | QA0   | QB0   | QC0   | QD0   | QE0   | QF0   | QH0   | QA0     | QH0 |
| Load        | H      | H  | H  | X    | X    | ↑   | X  | X  | a         | b     | c     | d     | e     | f     | g     | h     | a       | h   |

Рис. 3.4 – Функциональная таблица последовательного устройства

## 4. Выполнение работы

В соответствии с вариантом №9 задания, используя ранее описанные в лабораторных работах №1 и №2 устройства необходимо реализовать три варианта тестовых модулей.

### 4.1. Первый вариант тестового модуля для комбинационного устройства

-----  
-----  
-- Company:  
-- Engineer:  
--

```

-- Create Date: 29.09.2025 00:24:12
-- Design Name:
-- Module Name: tb_parallel_file - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--

```

---

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use STD.TEXTIO.ALL;
use IEEE.STD_LOGIC_TEXTIO.ALL;

entity tb_parallel_file is
end tb_parallel_file;

architecture Behavioral of tb_parallel_file is
    component parallel
        Port (
            not_1g, not_2g : in std_logic;
            a, b : in std_logic;
            c1_0, c1_1, c1_2, c1_3 : in std_logic;
            c2_0, c2_1, c2_2, c2_3 : in std_logic;
            y1, y2 : out std_logic
        );
    end component;

    -- signals
    signal not_1g, not_2g : std_logic;
    signal a, b : std_logic;
    signal c1_0, c1_1, c1_2, c1_3 : std_logic;
    signal c2_0, c2_1, c2_2, c2_3 : std_logic;
    signal y1, y2 : std_logic;

    file input_buf : text;

begin
    UUT: parallel
        port map (
            not_1g => not_1g,
            not_2g => not_2g,
            a => a,
            b => b,
            c1_0 => c1_0,
            c1_1 => c1_1,
            c1_2 => c1_2,
            c1_3 => c1_3,
            c2_0 => c2_0,
            c2_1 => c2_1,
            c2_2 => c2_2,
            c2_3 => c2_3,
            y1 => y1,
            y2 => y2
        )

```

```

);

stimulus: process
    variable read_input : line;
    variable line_num : integer := 0;
    variable not_1g_file, not_2g_file : std_logic;
    variable a_file, b_file : std_logic;
    variable c1_0_file, c1_1_file, c1_2_file, c1_3_file : std_logic;
    variable c2_0_file, c2_1_file, c2_2_file, c2_3_file : std_logic;
    variable y1_file, y2_file : std_logic;
begin
    file_open(input_buf, "D:\lab1_test.txt", read_mode);

    while not endfile(input_buf) loop
        line_num := line_num + 1;
        readline(input_buf, read_input);

        read(read_input, not_1g_file);
        read(read_input, not_2g_file);
        read(read_input, a_file);
        read(read_input, b_file);

        read(read_input, c1_0_file);
        read(read_input, c1_1_file);
        read(read_input, c1_2_file);
        read(read_input, c1_3_file);

        read(read_input, c2_0_file);
        read(read_input, c2_1_file);
        read(read_input, c2_2_file);
        read(read_input, c2_3_file);

        read(read_input, y1_file);
        read(read_input, y2_file);

        not_1g <= not_1g_file;
        not_2g <= not_2g_file;
        a <= a_file;
        b <= b_file;
        c1_0 <= c1_0_file;
        c1_1 <= c1_1_file;
        c1_2 <= c1_2_file;
        c1_3 <= c1_3_file;
        c2_0 <= c2_0_file;
        c2_1 <= c2_1_file;
        c2_2 <= c2_2_file;
        c2_3 <= c2_3_file;

        wait for 5 ns;

        if y1 /= y1_file then
            report "Line " & integer'image(line_num) & ": y1 not equal
to its file value" severity ERROR;
        end if;
        if y2 /= y2_file then
            report "Line " & integer'image(line_num) & ": y2 not equal
to its file value" severity ERROR;
        end if;
    end loop;

    file_close(input_buf);
    wait;

```

```

        end process;
    end Behavioral;

```

## 4.2. Второй вариант тестового модуля для комбинационного устройства

```

-----
-- Company:
-- Engineer:
--
-- Create Date: 29.09.2025 00:05:56
-- Design Name:
-- Module Name: tb_serial_file - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
-----

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use STD.TEXTIO.ALL;
use IEEE.STD_LOGIC_TEXTIO.ALL;

entity tb_serial_file is
end tb_serial_file;

architecture Behavioral of tb_serial_file is
    component serial
        Port (
            not_1g, not_2g : in std_logic;
            a, b : in std_logic;
            c1_0, c1_1, c1_2, c1_3 : in std_logic;
            c2_0, c2_1, c2_2, c2_3 : in std_logic;
            y1, y2 : out std_logic
        );
    end component;

    -- signals
    signal not_1g, not_2g : std_logic;
    signal a, b : std_logic;
    signal c1_0, c1_1, c1_2, c1_3 : std_logic;
    signal c2_0, c2_1, c2_2, c2_3 : std_logic;
    signal y1, y2 : std_logic;

    file input_buf : text;

begin
    UUT: serial
        port map (
            not_1g => not_1g,
            not_2g => not_2g,
            a => a,

```

```

        b => b,
        c1_0 => c1_0,
        c1_1 => c1_1,
        c1_2 => c1_2,
        c1_3 => c1_3,
        c2_0 => c2_0,
        c2_1 => c2_1,
        c2_2 => c2_2,
        c2_3 => c2_3,
        y1 => y1,
        y2 => y2
    );

stimulus: process
    variable read_input : line;
    variable line_num : integer := 0;
    variable not_1g_file, not_2g_file : std_logic;
    variable a_file, b_file : std_logic;
    variable c1_0_file, c1_1_file, c1_2_file, c1_3_file : std_logic;
    variable c2_0_file, c2_1_file, c2_2_file, c2_3_file : std_logic;
    variable y1_file, y2_file : std_logic;
begin
    file_open(input_buf, "D:\lab1_test.txt", read_mode);

    while not endfile(input_buf) loop
        line_num := line_num + 1;
        readline(input_buf, read_input);

        read(read_input, not_1g_file);
        read(read_input, not_2g_file);
        read(read_input, a_file);
        read(read_input, b_file);

        read(read_input, c1_0_file);
        read(read_input, c1_1_file);
        read(read_input, c1_2_file);
        read(read_input, c1_3_file);

        read(read_input, c2_0_file);
        read(read_input, c2_1_file);
        read(read_input, c2_2_file);
        read(read_input, c2_3_file);

        read(read_input, y1_file);
        read(read_input, y2_file);

        not_1g <= not_1g_file;
        not_2g <= not_2g_file;
        a <= a_file;
        b <= b_file;
        c1_0 <= c1_0_file;
        c1_1 <= c1_1_file;
        c1_2 <= c1_2_file;
        c1_3 <= c1_3_file;
        c2_0 <= c2_0_file;
        c2_1 <= c2_1_file;
        c2_2 <= c2_2_file;
        c2_3 <= c2_3_file;

        wait for 5 ns;

        if y1 /= y1_file then

```



```

        report "Line " & integer'image(line_num) & ": y1 not equal
to its file value" severity ERROR;
    end if;
    if y2 /= y2_file then
        report "Line " & integer'image(line_num) & ": y2 not equal
to its file value" severity ERROR;
    end if;
end loop;

    file_close(input_buf);
    wait;
end process;
end Behavioral;

```

### 4.3. Тестовый модуль для функционального узла последовательного типа

```

-----
----
-- Company:
-- Engineer:
--
-- Create Date: 29.09.2025 00:31:04
-- Design Name:
-- Module Name: tb_registers_file - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
-----
----

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use std.textio.all;
use ieee.std_logic_textio.all;

entity tb_registers is
end tb_registers;

architecture Behavioral of tb_registers is
    component registers
        Port (
            S0, S1, SR, SL, CLK, CLR, OE1, OE2 : in std_logic;
            A_in, B_in, C_in, D_in, E_in, F_in, G_in, H_in : in std_logic;
            A_out, B_out, C_out, D_out, E_out, F_out, G_out, H_out : out
std_logic
        );
    end component;

    -- Signals
    signal S0, S1, SR, SL, CLK, CLR, OE1, OE2 : std_logic := '0';
    signal A_in, B_in, C_in, D_in, E_in, F_in, G_in, H_in : std_logic := '0';
    signal A_out, B_out, C_out, D_out, E_out, F_out, G_out, H_out : std_logic;

```

```

file input_buf : text;

begin
    UUT: registers
        port map(
            S0 => S0, S1 => S1, SR => SR, SL => SL, CLK => CLK, CLR => CLR,
            OE1 => OE1, OE2 => OE2,
            A_in => A_in, B_in => B_in, C_in => C_in, D_in => D_in,
            E_in => E_in, F_in => F_in, G_in => G_in, H_in => H_in,
            A_out => A_out, B_out => B_out, C_out => C_out, D_out => D_out,
            E_out => E_out, F_out => F_out, G_out => G_out, H_out => H_out
        );

    stimulus: process
        variable read_input : line;
        variable line_num : integer := 0;
        variable S0_file, S1_file, SR_file, SL_file, CLK_file, CLR_file :
std_logic;
        variable OE1_file, OE2_file : std_logic;
        variable A_in_file, B_in_file, C_in_file, D_in_file : std_logic;
        variable E_in_file, F_in_file, G_in_file, H_in_file : std_logic;
        variable A_out_file, B_out_file, C_out_file, D_out_file : std_logic;
        variable E_out_file, F_out_file, G_out_file, H_out_file : std_logic;
    begin
        file_open(input_buf, "D:\lab2_test.txt", read_mode);

        while not endfile(input_buf) loop
            line_num := line_num + 1;
            readline(input_buf, read_input);

            read(read_input, S0_file); read(read_input, S1_file);
            read(read_input, SR_file); read(read_input, SL_file);
            read(read_input, CLK_file); read(read_input, CLR_file);
            read(read_input, OE1_file); read(read_input, OE2_file);
            read(read_input, A_in_file); read(read_input, B_in_file);
            read(read_input, C_in_file); read(read_input, D_in_file);
            read(read_input, E_in_file); read(read_input, F_in_file);
            read(read_input, G_in_file); read(read_input, H_in_file);

            read(read_input, A_out_file); read(read_input, B_out_file);
            read(read_input, C_out_file); read(read_input, D_out_file);
            read(read_input, E_out_file); read(read_input, F_out_file);
            read(read_input, G_out_file); read(read_input, H_out_file);

            S0 <= S0_file; S1 <= S1_file; SR <= SR_file; SL <= SL_file;
            CLK <= CLK_file; CLR <= CLR_file; OE1 <= OE1_file; OE2 <= OE2_file;
            A_in <= A_in_file; B_in <= B_in_file; C_in <= C_in_file; D_in <=
D_in_file;
            E_in <= E_in_file; F_in <= F_in_file; G_in <= G_in_file; H_in <=
H_in_file;

            wait for 5 ns;

            -- проверка выходов
            if A_out /= A_out_file then
                report "Line " & integer'image(line_num) & ": A_out mismatch.
Expected=" &
std_logic'image(A_out_file) & " Got=" &
std_logic'image(A_out) severity ERROR;
            end if;
            if B_out /= B_out_file then

```

```

        report "Line " & integer'image(line_num) & ": B_out mismatch.
Expected=" &
        std_logic'image(B_out_file) & " Got=" &
std_logic'image(B_out) severity ERROR;
    end if;
    if C_out /= C_out_file then
        report "Line " & integer'image(line_num) & ": C_out mismatch.
Expected=" &
        std_logic'image(C_out_file) & " Got=" &
std_logic'image(C_out) severity ERROR;
    end if;
    if D_out /= D_out_file then
        report "Line " & integer'image(line_num) & ": D_out mismatch.
Expected=" &
        std_logic'image(D_out_file) & " Got=" &
std_logic'image(D_out) severity ERROR;
    end if;
    if E_out /= E_out_file then
        report "Line " & integer'image(line_num) & ": E_out mismatch.
Expected=" &
        std_logic'image(E_out_file) & " Got=" &
std_logic'image(E_out) severity ERROR;
    end if;
    if F_out /= F_out_file then
        report "Line " & integer'image(line_num) & ": F_out mismatch.
Expected=" &
        std_logic'image(F_out_file) & " Got=" &
std_logic'image(F_out) severity ERROR;
    end if;
    if G_out /= G_out_file then
        report "Line " & integer'image(line_num) & ": G_out mismatch.
Expected=" &
        std_logic'image(G_out_file) & " Got=" &
std_logic'image(G_out) severity ERROR;
    end if;
    if H_out /= H_out_file then
        report "Line " & integer'image(line_num) & ": H_out mismatch.
Expected=" &
        std_logic'image(H_out_file) & " Got=" &
std_logic'image(H_out) severity ERROR;
    end if;

    end loop;

    file_close(input_buf);
    wait;
end process;

end Behavioral;

```

## 5. Выполнение автоматической проверки работы устройств

Результаты проведения проверки работы первого варианта тестового модуля комбинационного устройства представлены на рисунке 5.1 и 5.2.

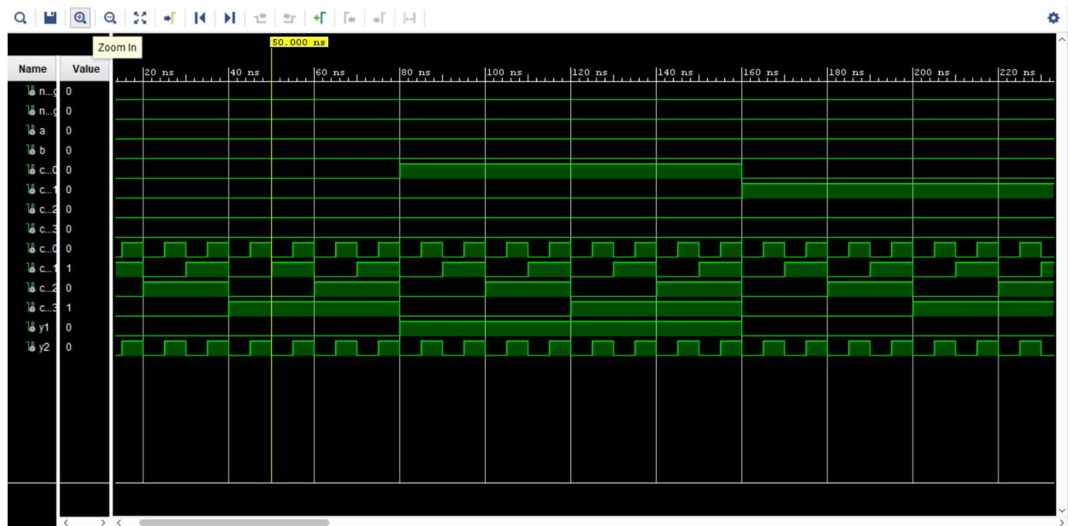


Рис. 5.1 – Результат первого варианта

```
source tb_serial_file.tcl
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [llength [get_objects]] > 0 } {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#     send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->New Waveform Co
#   }
# }
WARNING: Simulation object /tb_serial_file/input_buf was not traceable in the design for the following reason:
Vivado Simulator does not yet support tracing of VHDL variables.
# run 1000ns
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_serial_file_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:02 ; elapsed = 00:00:06 . Memory (MB): peak = 1012.691 ; gain = 6.078
```

Рис. 5.2 – Содержимое консоли во время выполнения первого варианта

Результаты проведения проверки работы второго варианта тестового модуля комбинационного устройства представлены на рисунке 5.3 и 5.4.

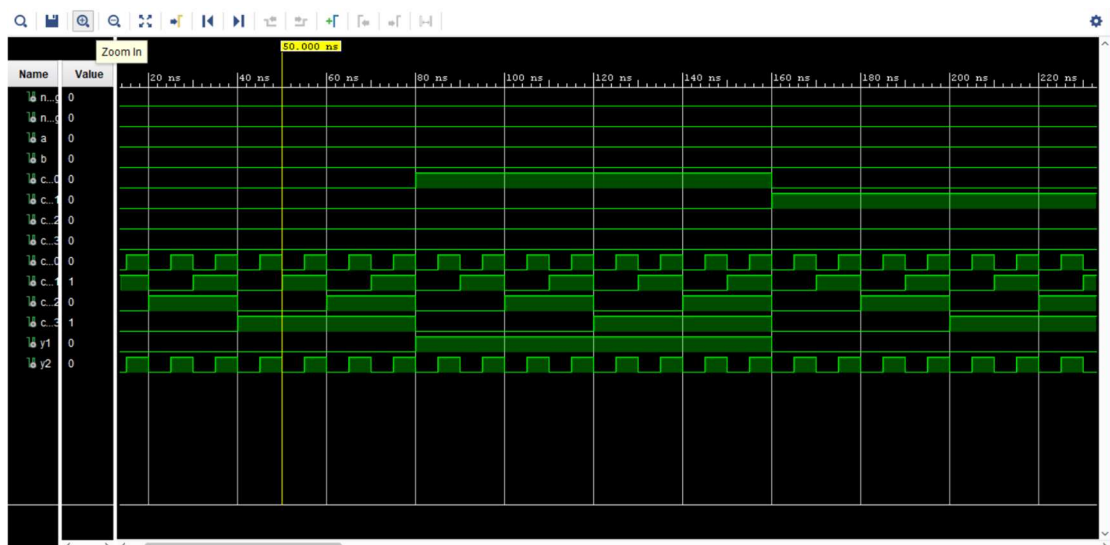


Рис. 5.3 – Результат второго варианта

```

source tb_parallel_file.tcl
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [length [get_objects]] > 0 } {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#     send_msg_id AddWave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->New Waveform'"
#   }
# }
WARNING: Simulation object /tb_parallel_file/input_buf was not traceable in the design for the following reason:
Vivado Simulator does not yet support tracing of VHDL variables.
# run 1000ns
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_parallel_file_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns

```

Рис. 5.4 – Содержимое консоли во время выполнения второго варианта

Результаты проведения проверки работы тестового модуля последовательного устройства представлены на рисунке 5.5 и 5.6.

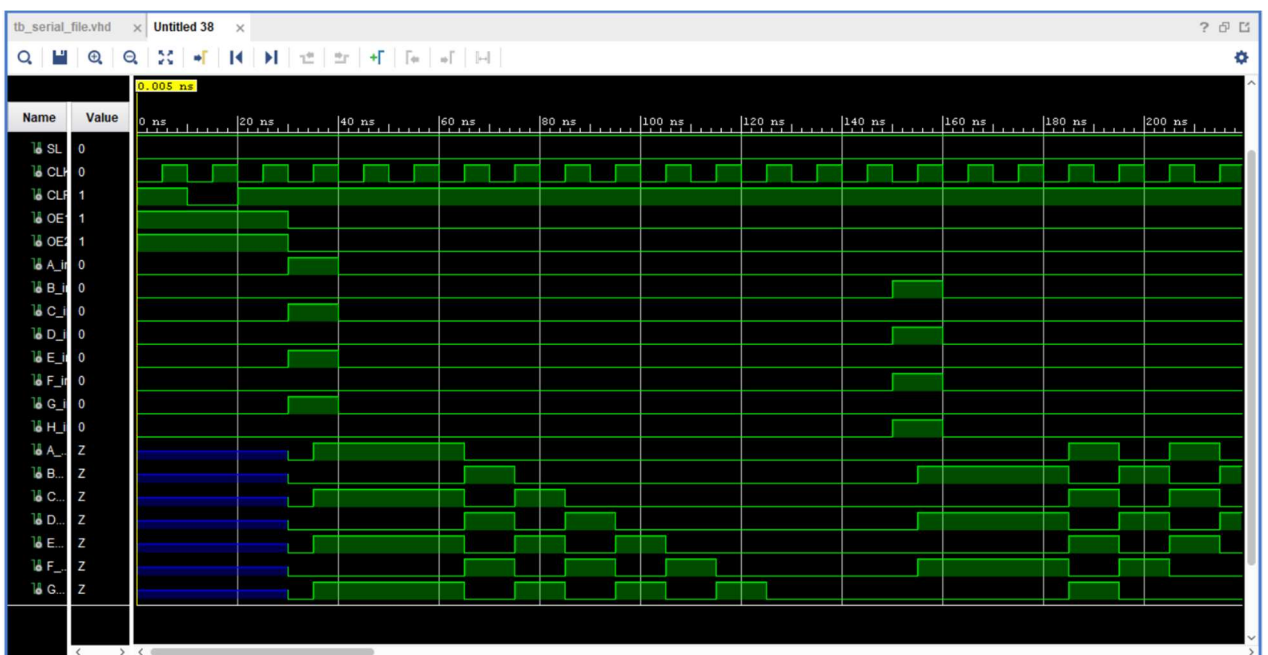


Рис. 5.5 – Результат последовательного устройства

```

source tb_registers.tcl
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [length [get_objects]] > 0 } {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#     send_msg_id AddWave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->New Waveform'"
#   }
# }
WARNING: Simulation object /tb_registers/input_buf was not traceable in the design for the following reason:
Vivado Simulator does not yet support tracing of VHDL variables.
# run 1000ns
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_registers_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns

```

Рис. 5.6 – Содержимое консоли во время выполнения последовательного устройства

## **6. Выводы**

В процессе выполнения работы были приобретены навыки разработки тестовых модулей на языке VHDL.

Полученные навыки были применены для решения задач, возникших в ходе работы: разработки модуля генерации входных и эталонных тестовых воздействий для тестируемых устройств; разработки модулей тестирования комбинационного устройства; проведения автоматической проверки корректности работы комбинационного устройства; разработки модуля тестирования функционального узла последовательного типа; проведения автоматической проверки корректности работы функционального узла последовательного типа.